Security Assessment: Assistant Hub (rmassistanthub.io)

Nov 8, 2026 · @Jeff Dingess

A source-assisted security review of the Assistant Hub codebase and a read-only external configuration check of the live site. No critical or high-severity findings: the money and authentication paths are well-built, and the one real external weakness is a defense-in-depth gap in the Content Security Policy.

Scope and method

The review combined static code analysis of the application source with a non-intrusive check of the live site's configuration.

- **Target:** the `redman4220/assistant-hub` codebase (Bun + Hono, vanilla SPA) and the production site at rmassistanthub.io (Railway + Cloudflare).
- **In scope:** payment and money paths (Stripe webhook, USDC/x402 verification, replay prevention), authentication and authorization, SQL injection, server-side request forgery (SSRF), CORS and credential handling, committed secrets, and live security headers and TLS.
- **Out of scope:** dynamic fuzzing, authenticated attack traffic, and load testing against production. Those exercise Railway and Cloudflare infrastructure and need the provider's authorization, so they were deliberately not performed.
- **Authorization:** the assessor is the sole developer and owner of the application and its infrastructure.

This is a white-box review: findings reference the actual code rather than inferring behavior from the outside, which is higher-signal than black-box scanning and carries no risk to the running service.

Results summary

Every area tested held up. The findings that remain are low-severity or informational.

Area	Verdict	Basis
Stripe webhook	Pass	Async signature verification, paid-status gate, claim-before-grant with release on failure, dedup on session and invoice id, returns 500 so Stripe retries
USDC / x402 verification	Pass	Transfer read back from a Base RPC node; only a verified status grants; exact recipient, amount and 30-minute age enforced
Replay / double-spend	Pass	Database UNIQUE constraint on the transaction hash; the check-then-insert race is caught by the constraint; database-down nets fail-closed
SQL injection	Pass	No interpolated values; the one dynamic-column builder uses server-set literal keys, not attacker input
SSRF	Pass	Variable-URL fetches use internal constants or run client-side; none takes a user-supplied host
Admin authorization	Pass	Admin routes re-load the user and check role against current database state, not the token claim
CORS + credentials	Pass	Credentialed CORS only echoes origins on a fixed allowlist; credentials are stripped whenever the origin is a wildcard
Committed secrets	Pass	No private keys or live API keys in tracked files
External headers / TLS	Pass	HSTS with preload, tight CSP, anti-framing, nosniff, security.txt present, Cloudflare fronting

Findings

Four items are worth acting on. None is an open door; the two low-severity items are defense-in-depth improvements, and two are informational.

1. Low — CSP allows `unsafe-inline` for scripts and styles. The Content Security Policy permits inline scripts and styles, so if an XSS flaw were ever introduced, the CSP would not blunt it. The rest of the policy is tight: `object-src 'none'`, `frame-ancestors 'none'`, `base-uri 'self'`, `form-action 'self'`, `upgrade-insecure-requests`.

Remediation: move inline handlers and styles to nonces or external files. This is substantial work on a large single-page app, so it is a long-term hardening item rather than an urgent fix.

2. Low — Authentication token stored in `localStorage`. The session token is kept in `localStorage`, which JavaScript can read, so an XSS flaw could exfiltrate it. The tight CSP above is the main mitigation. Remediation: move the token to an `HttpOnly` cookie, which scripts cannot read. Trade-off: cookies introduce CSRF considerations, so this is a design decision, not a quick switch.

3. Informational — USDC payments are first-come by design. On-chain verification confirms a transfer happened but does not bind the transaction to the caller, so whoever submits a fresh hash first redeems it. The code documents this intentionally. It is acceptable for the current model and worth re-examining before expanding USDC payment rails; closing it requires the payer to sign the request (EIP-3009 authorization).

4. Informational — The admin-authorization helper is duplicated across route files. Each copy is correct, but duplication invites drift: a future change could update one file and miss another. Remediation: consolidate into a single shared middleware.

External posture

The live site's response headers were read with a single ordinary request. The configuration is strong.

Control	Observed
HTTPS / TLS	HTTP/2 over TLS, Cloudflare-terminated
HSTS	`max-age=31536000; includeSubDomains; preload`
Content-Security-Policy	Present and tight: `default-src 'self'`, `object-src 'none'`, `frame-ancestors 'none'`, `base-uri 'self'`, `form-action 'self'`, `upgrade-insecure-requests` (weakened only by `unsafe-inline` on scripts and styles — see finding 1)
Clickjacking	`X-Frame-Options: DENY` plus `frame-ancestors 'none'`
MIME sniffing	`X-Content-Type-Options: nosniff`

Control	Observed
Referrer-Policy	`strict-origin-when-cross-origin`
Permissions-Policy	camera, microphone, geolocation and FLoC all disabled
Disclosure	`/.well-known/security.txt` present (HTTP 200) — a mark of security maturity
robots.txt	Agent surfaces explicitly allowed ahead of a broad `/api/` disallow, by design

The presence of a security.txt and a hand-tuned CSP indicates the site was configured with security in mind rather than left on framework defaults.

Conclusion

The hardest parts of a crypto application to get right — payment settlement, replay prevention, and authorization — are the parts this codebase handles most carefully. On-chain payments are verified against a node rather than an explorer, only a fully verified status grants access, a database constraint backs replay prevention, and authorization decisions read live user state instead of trusting a token's claims. The single real external weakness is the `unsafe-inline` allowance in an otherwise strict CSP, and that is defense-in-depth, not an exposed vulnerability.

Skills demonstrated

- **White-box application security review** across payment, authentication, injection, SSRF and CORS classes
- **Threat modeling** of money paths: replay, double-spend, race conditions, and webhook forgery
- **Secure configuration assessment** of HTTP security headers, TLS and CSP
- **Risk-based triage:** separating exploitable flaws from defense-in-depth gaps and documenting severity and remediation for each

No critical or high-severity findings were identified. Two low-severity hardening items and two informational notes are tracked above.